

Día 1: MyBingo!

Requisitos previos:  Intermedio

Capítulo 1: Eventos.....	2
Capítulo 2: Instalaciones.....	3
QT Designer.....	3
PySide6.....	3
Capítulo 3: Generando nuestro proyecto.....	3
Botones.....	5
Cuadros de texto.....	5
Ventana de construcción.....	6
Generación del proyecto.....	6
Capítulo 4: La lógica del programa.....	10
Conexión de botones.....	10
Generación de los números.....	11
Guardar el cartón.....	12
Conclusiones.....	13

¡Hola a tod@s!

En este tutorial, vamos a crear una aplicación utilizando *Python* + la librería *PySide6* y el constructor de UI *QT Designer*.

La idea principal de este aplicativo será conocer de alguna manera las bases de la **programación de eventos** construyendo un programa que nos permita generar en nuestro ordenador un archivo de texto con el cartón de un bingo.

Antes de continuar con el tutorial, hago un pequeño inciso para presentarme. Mi nombre es *Cristopher Méndez Cervantes* y actualmente me dedico a la programación de backend utilizando el framework de *Odoo* para crear aplicaciones funcionales de entorno ERP/CRM.

Llevo más de 2 años programando profesionalmente, siendo mi lenguaje natural Python, y actualmente me dedico en mi tiempo libre a compartir píldoras informativas, fomentando la creación de un ecosistema de programadores con amor por el conocimiento libre y accesible para todos.

¡Pero basta de presentaciones! Ha llegado el momento de emprender un viaje hacia un nuevo mundo de luz y color, ya que por fin pasaremos de scriptear en consola a generar nuestras propias interfaces.

Capítulo 1: Eventos

Toda aplicación que aprecie de una interfaz gráfica requiere de un flujo constante de conseguir información para luego mostrarla al usuario. Para que esto ocurra, ¡algo tiene que pasar!

Normalmente y cuando trabajamos en el diseño de interfaces, tenemos que pensar en qué cosas harán que cierto comportamiento de nuestro programa ocurra.

Para que nos entendamos mejor, pensemos en un formulario típico. Cada campo es rellenado por el usuario, y luego es procesado cuando pincha en un botón. ¡Es en ese momento cuando ocurre la **magia**!

El botón desencadena un evento o acción. Esa acción no es más que código que se ejecuta cuando el usuario hace un tipo de evento (de clic).

Traducido al cristiano de los programadores, un evento ejecuta una función o método de una clase, que describirá el comportamiento natural de dicho evento.

Usuario > Acción > Procesamiento de datos > Renderización

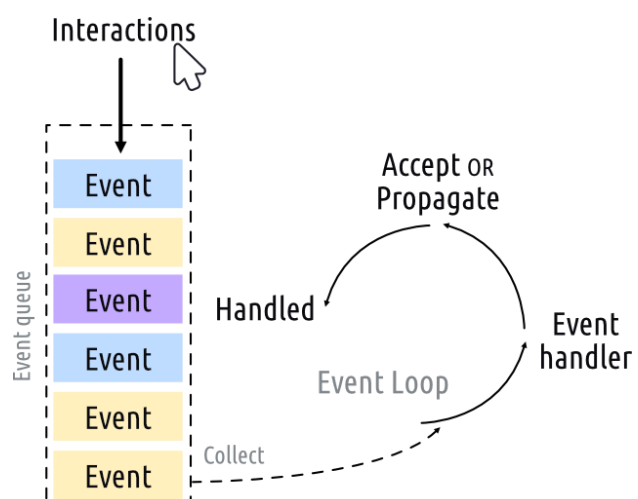
Las interfaces muestran el resultado de los eventos, y muchos eventos también dependen de estados de la interfaz.

Antes de crear nuestra primera ventana, hay que introducir algunos conceptos clave sobre cómo se organizan las aplicaciones en el mundo Qt.

El corazón de toda aplicación hecha con Qt es la clase QApplication. Cada aplicación necesita un - y *sólo un* - objeto QApplication para funcionar. Este objeto contiene el **bucle de eventos** de tu aplicación - el bucle central que controla toda la interacción del usuario con la pantalla. Cada interacción con la aplicación, ya sea pulsar una tecla, hacer clic con el ratón o mover el ratón, genera un evento que se coloca en la cola de eventos.

En el bucle de eventos, la cola se comprueba en cada iteración y si se encuentra un evento en espera, el evento y el control se pasan al manejador de eventos específico para el evento. El manejador de eventos se ocupa del evento, luego pasa el control de nuevo al bucle de eventos para esperar más eventos. **Sólo hay un bucle de eventos en ejecución por aplicación.**

A continuación, y sin más dilación, es hora de construir nuestra primera aplicación con QT Designer, así que primero lo que vamos a hacer es instalar las dependencias necesarias para poner en marcha el proyecto.



Capítulo 2: Instalaciones

QT Designer

QT Designer es una aplicación gratuita que nos permite construir las diferentes ventanas de nuestra aplicación de una manera ágil. Nos permite crear y arrastrar widgets (botones, cuadros de texto...), ahorrando tiempo a cambio de sacrificar gran parte del control sobre nuestra aplicación.

En enlace de descarga se puede encontrar [aquí](#).

PySide6

PySide6 es la librería oficial del framework que envuelve a Qt. Puede que aún no hayáis trabajado con frameworks y sea esta vuestra primera vez, así que me limitaré a ser breve y conciso; un framework es una especie de marco de trabajo que nos permite desarrollar aplicaciones siguiendo unas normas, de manera rápida y ordenada.

Pensemos en una librería que contiene TODO lo necesario para poder poner en marcha nuestros proyectos, siguiendo a menudo ciertos patrones de comportamiento y cierta sintaxis que debemos aprender y respetar. Esta librería es PySide6, y la utilizaremos para dar vida a nuestra aplicación.

Todo lo referente a la documentación de la librería se puede encontrar en el siguiente [enlace](#).

Para instalar la aplicación, en nuestra terminal de VSC ejecutaremos el siguiente comando:

```
pip install PySide6
```

Con esto se instalará la librería junto a todas sus dependencias. Esto lo podemos hacer en local en nuestra máquina o dentro de un [entorno virtual](#).

Capítulo 3: Generando nuestro proyecto

Vamos a escribir un programa en Python que simule la generación de cartones para jugar a una versión reducida y rápida del Bingo.

En el Bingo, un jugador tiene uno o varios cartones, donde cada uno representa una matriz de números. Un locutor va sacando eventualmente una bola de un bombo y recita en alto el número que ha salido. Si el cartón tiene dicho número, se tacha. Gana el jugador que primero cante bingo tachando todos sus números del/los cartón/es.

Hay muchas versiones del bingo, pero nosotros vamos a generar la nuestra, siendo un bingo exprés cuyos cartones se componen de:

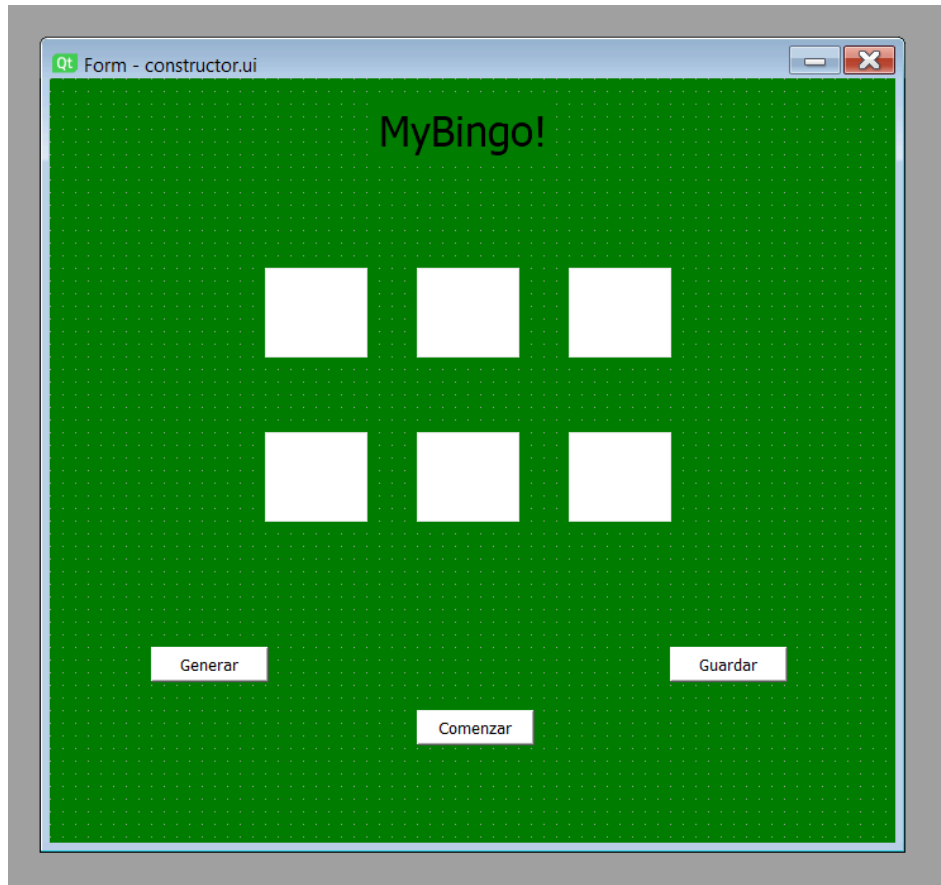
1. 6 casillas.
2. Cada casilla contiene un número aleatorio SIN REPETIR del 1-50 incluyendo ambos extremos (e. [1, 50]).

Para ello, necesitamos pensar primero en la generación de nuestra interfaz... ya luego pensaremos en la lógica.

Lo primero que tenemos que pensar a la hora de construir una interfaz es pensar en qué vamos a necesitar y qué elementos intervienen. En nuestro caso, pensaremos primero en:

1. Tipo de ventana.
2. Los widgets o componentes que irán dentro.

Para este primer día, queremos conseguir el siguiente objetivo:



¿Podemos echar un vistazo a los elementos que intervienen?

Hay tres claros elementos en esta interfaz:

- a) La ventana misma, que contiene un fondo de color verde.
- b) Los tres botones de acción.
- c) Los paneles o cuadros de texto.

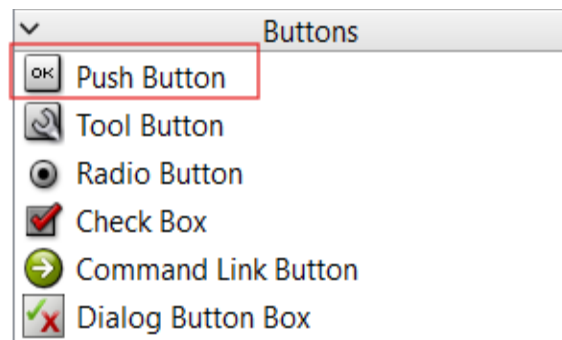
A continuación, desglosamos la semántica de cada elemento gráfico:

Botones

Los botones que hemos elegido emanan del componente Push Button. Un elemento Push Button genera un botón gráfico que luego conectaremos a nuestras diferentes funciones que harán la lógica de los mismos al pulsarlos.

En nuestro caso queremos **3 botones**.

En primer lugar tenemos el botón *Comenzar*. Todos los otros componentes están por defecto deshabilitados en nuestro programa (enabled = False).

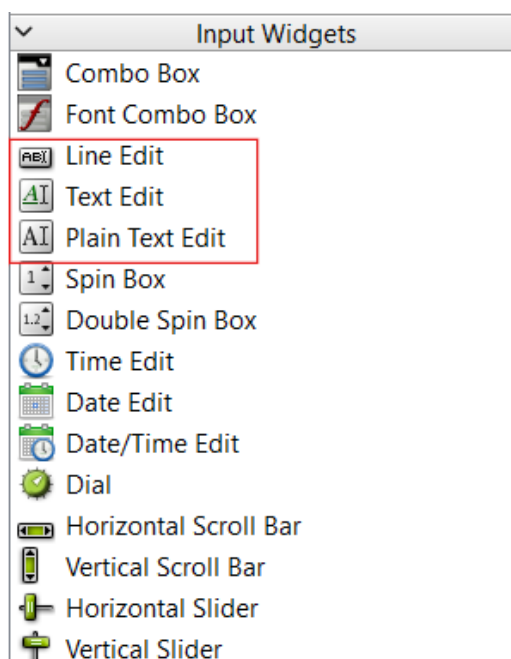


Con un clic en *Comenzar*, los botones *Generar* y *Guardar* se habilitarán, y el texto en el botón *Comenzar* cambiará a *Detener*.

El botón *Generar*, al hacer clic, generará 6 números no repetidos aleatorios y repartidos entre los diferentes cuadros de texto.

El botón *guardar* guardará de manera local nuestros 6 números desplegados en la pantalla en nuestra raíz de proyecto como formato de texto (txt).

Cuadros de texto



Los cuadros de texto son la principal entrada de datos de un usuario. Sin embargo, y para darle un poco la vuelta al ejercicio, serán el objeto que nos permitirá observar resultados en nuestro programa (los números del cartón).

Como se puede ver, hay tres tipos de cuadros de texto.

Line Edit es el texto más básico y normalmente se utiliza para la introducción de datos típica de

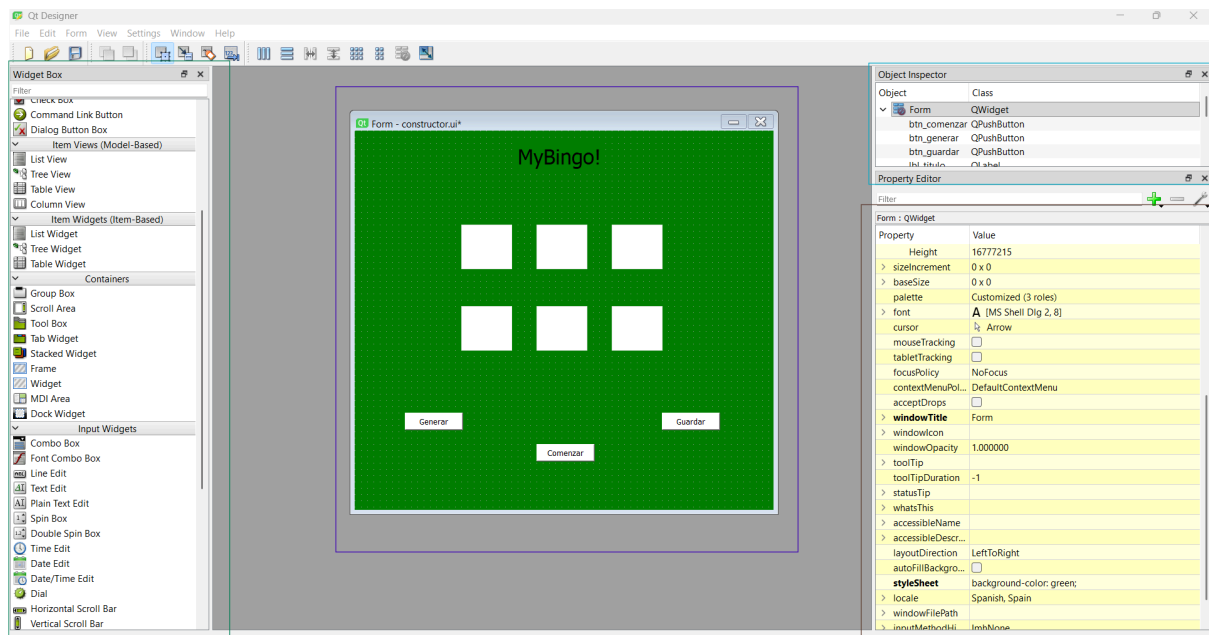
formularios. Acepta poco formateo y no permite saltos de línea.

Text Edit es el que se utilizará para el proyecto. Permite saltos de línea y un formato más enriquecido, estilo Word.

Plain Text Edit es para grandes volúmenes de información, siendo ligero y sólido.

¿Cómo sacamos estos componentes? Vamos a darnos un paseo por la interfaz.

Ventana de construcción



En diferentes colores, podemos ver la composición de una ventana de tipo Widget en QT Designer. En **azul celeste**, el Object Inspector, que nos permite asociar los componentes de interfaz con su clase instanciada en PySide6. Podemos gestionar rápidamente el nombre de los objetos, ayudándonos a la hora de generar el código fuente en terminal.

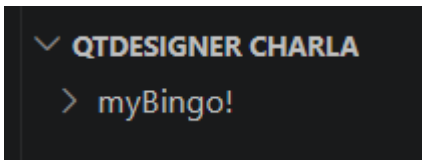
En **marrón**, podemos ver el rectángulo de propiedades. Cada elemento tiene sus propias propiedades y podemos jugar con ellas, instanciando o eliminando aquellas propiedades que nos interesen.

En **verde** podemos ver el rectángulo con todos los drag-and-drop widgets que podemos utilizar para construir nuestra interfaz, separados y catalogados por sus familias o tipos.

Por último, y en **morado**, lo que nos interesa, el resultado final, nuestra interfaz

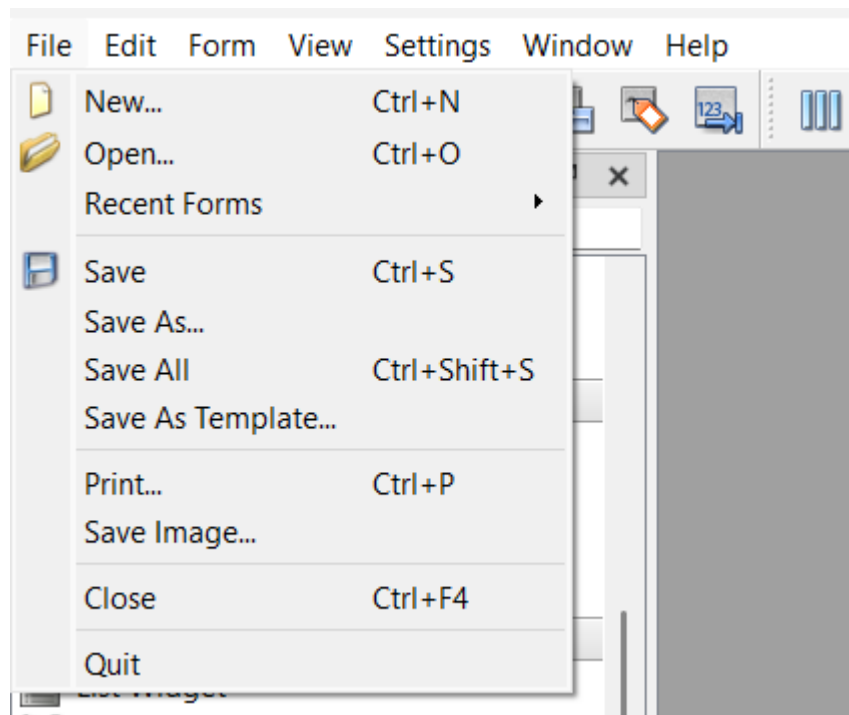
Generación del proyecto

En VSC code vamos a crearnos una estructura de directorios arbitraria. Insisto, ¡cualquiera vale! Solo necesitamos tener nuestro espacio de trabajo acomodado.

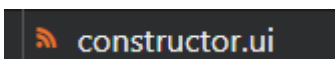


Dentro de la carpeta que hemos elegido para nuestro proyecto, tenemos que cargar el archivo .ui de nuestra ventana de QT Designer. Para ello debemos ir a *File > Save As...*

y guardar el archivo en nuestra carpeta de proyecto.



Se nos ha generado lo siguiente:



Se trata de un archivo XML que contiene todos los componentes gráficos necesarios para generar el contexto y la forma de nuestra ventana.

Con este archivo, y gracias a un comando, podemos generar nuestro archivo de python que contiene la clase de nuestra ventana para empezar a escribir toda la lógica que la envuelve.

Para ello, escribimos en terminal:

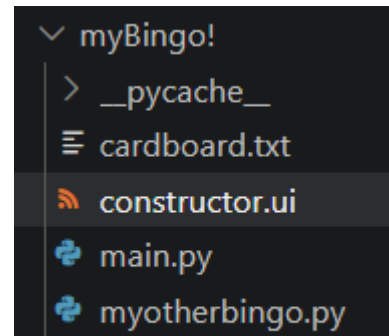
```
pyside6-uic constructor.ui -o
nombre_de_tu_ventana.py
```

El resultado será la obtención de la transcripción del archivo XML a un archivo de Python con la clase de la ventana:

En mi caso, y como se puede observar, el nombre de la ventana es myotherbingo.py.

Podéis inspeccionar el código, pero yo os adelanto que el código es puramente la transcripción de lo que habéis generado en ventana.

Volveremos al código más adelante. Ahora quiero hablar un poco acerca del archivo main.py, que será el archivo director que ejecutará el bucle de eventos.



```
1  import sys
2
3  from PySide6.QtWidgets import QApplication, QWidget
4  from myotherbingo import MyBingo
5
6  class MainWindow(QWidget, MyBingo):
7      def __init__(self, parent=None):
8          super(MainWindow, self).__init__(parent)
9          self.setupUi(self)
10
11
12  def main():
13      app = QApplication(sys.argv)
14      window = MainWindow()
15      window.show()
16      app.exec()
17
18  if __name__ == '__main__':
19      main()
```

La clase **MainWindow**, que hereda de QWidget (Comportamiento de la ventana) y MyBingo (El diseño, la clase generada por el comando). En el main instanciamos la aplicación y conectamos la clase, arrancando el bucle de eventos.

Este será el archivo que tendremos que ejecutar para instanciar nuestra ventana.

Volvamos ahora con nuestra clase generada:

```
1 import random
2
3 from PySide6.QtCore import (QCoreApplication, QDate, QDateTime, QLocale,
4     QMetaObject, QObject, QPoint, QRect,
5     QSize, QTime, QUrl, Qt)
6 from PySide6.QtGui import (QBrush, QColor, QConicalGradient, QCursor,
7     QFont, QFontDatabase, QGradient, QIcon,
8     QImage, QKeySequence, QLinearGradient, QPainter,
9     QPalette, QPixmap, QRadialGradient, QTransform)
10 from PySide6.QtWidgets import (QApplication, QLabel, QPushButton, QSizePolicy,
11     QTextEdit, QWidget)
12
13 class MyBingo(object):
14     def setupUi(self, Form):
15         if not Form.setObjectName():
16             Form.setObjectName(u"Form")
17         Form.resize(668, 605)
18         Form.setStyleSheet(u"background-color: green;")
19         self.t_edit_1 = QTextEdit(Form)
20         self.t_edit_1.setObjectName(u"t_edit_1")
21         self.t_edit_1.setEnabled(False)
22         self.t_edit_1.setGeometry(QRect(170, 150, 81, 71))
23         self.t_edit_1.setAutoFillBackground(False)
24         self.t_edit_1.setStyleSheet(u"background-color: white;color:black;")
25         self.t_edit_2 = QTextEdit(Form)
26         self.t_edit_2.setObjectName(u"t_edit_2")
27         self.t_edit_2.setEnabled(False)
28         self.t_edit_2.setGeometry(QRect(290, 150, 81, 71))
29         self.t_edit_2.setAutoFillBackground(False)
30         self.t_edit_2.setStyleSheet(u"background-color: white;color:black;")
31         self.t_edit_3 = QTextEdit(Form)
32         self.t_edit_3.setObjectName(u"t_edit_3")
33         self.t_edit_3.setEnabled(False)
34         self.t_edit_3.setGeometry(QRect(410, 150, 81, 71))
35         self.t_edit_3.setAutoFillBackground(False)
36         self.t_edit_3.setStyleSheet(u"background-color: white;color:black;")
37         self.t_edit_4 = QTextEdit(Form)
```

Contiene todas y cada una de las configuraciones aportadas para poder conformar la ventana que hemos creado desde el programa de diseño. Aquí es donde empezaremos a picar código y trabajar la lógica de nuestra aplicación, DENTRO de la clase.

Capítulo 4: La lógica del programa

Conexión de botones

Hacer que los botones ejecuten lógica será nuestra primera labor. Para ello, tenemos que conocer el concepto de **slot**.

Qt utiliza un mecanismo llamado señales para permitirte reaccionar a eventos como que el usuario pulse un botón. Para ello debemos atender la señal **.clicked** y debemos conectarlo con el código que se quiera ejecutar mediante el método **.connect**

Para demostrar el funcionamiento, atenderemos al siguiente código:

```
# ===== variables and constants ===== #

self.is_stopped = True
self.bingo_card = []

# ===== main events ===== #

def start_or_pause():
    '''This slot activates/disables the other buttons.'''
    if self.is_stopped:
        self.btn_comenzar.setText("Detener")
        self.is_stopped = False
        self.btn_generar.setEnabled(True)
        return
    self.btn_comenzar.setText("Comenzar")
    self.is_stopped = True
    self.btn_generar.setEnabled(False)
    return
```

El método **start_or_pause()** será la lógica que ejecutará el botón cuyo texto reza "Comenzar".

Si queremos conectar el botón, tenemos que posteriormente vincularlo al método:

```
# ===== main slots ===== #

self.btn_comenzar.clicked.connect(start_or_pause) #this button displays/sets up the environment.
self.btn_generar.clicked.connect(generate_numbers) #this button generates the numbers.
self.btn_guardar.clicked.connect(save_cardboard) #this button saves the cardboard.
```

Con esto, el programa ya puede empezar a funcionar como queremos. A continuación, tenemos que generar los números. Para ello, nos valdremos de la librería random.

Generación de los números

Para generar los números, utilizaremos el método **.randint(extremo izquierdo, extremo derecho)**.

```
1  ∨ import random
2
```

```
def generate_numbers():
    '''This slot mainly generates a 2x3 numeric matrix of 1-50 range random numbers without any repetition.'
    if self.bingo_card:
        self.bingo_card.clear()

    for _ in range(6):
        while True:
            my_number = str(random.randint(1,50))
            if my_number not in self.bingo_card:
                self.bingo_card.append(my_number)
                break
    self.t_edit_1.setText(self.bingo_card[0])
    self.t_edit_2.setText(self.bingo_card[1])
    self.t_edit_3.setText(self.bingo_card[2])
    self.t_edit_4.setText(self.bingo_card[3])
    self.t_edit_5.setText(self.bingo_card[4])
    self.t_edit_6.setText(self.bingo_card[5])
    self.btn_guardar.setEnabled(True)
```

Es buena práctica limpiar los registros si ya teníamos algunos. Al dar clic, siempre vamos a generar nuevos números, es por eso que hay que cuidar bien la interfaz.

Este algoritmo sencillamente genera una matriz numérica plana de 6 números del 1-50 sin repetir en una lista. ¡Ojo! Los números deben ser tratados como String, si no, la aplicación crashearán en tiempo de ejecución.

Una vez generados los números, el método `.setText` sobre un componente `textEdit` sobrescribe la información sobre cada panel. Las asignaciones están hardcodeadas para uso explicativo.

Reto: ¿Cómo podríamos refactorizar este código?

Guardar el cartón

Para guardar el cartón tendremos primeramente que recordar cómo se escribe un fichero plano en Python.

```
def save_cardboard():
    if not self.bingo_card:
        pass
    # ===== GENERATE THE CARDBOARD LAYOUT ===== #
    with open('cardboard.txt', 'w') as f:
        cardboard = ""
        for number in self.bingo_card:
            cardboard += f"| {number} |"
        f.write(cardboard)
```

El manejador de contexto `with open` asegura cerrar el puntero una vez el cartón es generado.

Conclusiones

Con esta primera aproximación podemos ver claramente cómo funciona la programación de eventos, además de como generar nuestros propios programas básicos desde 0.

En próximas entregas, aprenderemos a utilizar otros componentes, nuevas técnicas y métodos y pasar contextos entre ventanas.

Si te ha gustado el repositorio, por favor, dale una star. ¡Me ayuda a mejorar y transmitir conocimiento a más gente!